

1649 lines (1452 loc) · 73.8 KB

```

1  import os
2  import time
3  import re
4  import shutil
5  import tempfile
6  from typing import List, Literal, Optional, Any, Set, Tuple, Dict
7
8  from fastapi import FastAPI, HTTPException, Request, UploadFile, File, status
9  from fastapi.middleware.cors import CORSMiddleware
10 from fastapi.staticfiles import StaticFiles
11 from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
12 from pydantic import BaseModel
13 from dotenv import load_dotenv
14
15 import pandas as pd
16 import chromadb
17 from openai import OpenAI
18
19 from datetime import datetime
20 from zoneinfo import ZoneInfo
21
22 # =====
23 # ENVIRONMENT
24 # =====
25 load_dotenv()
26
27 OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
28 OPENAI_MODEL = os.getenv("OPENAI_MODEL", "gpt-4o-mini")
29 OPENAI_EMBED_MODEL = os.getenv("OPENAI_EMBED_MODEL", "text-embedding-3-small")
30
31 # translation model can be overridden if you want a cheaper model
32 TRANSLATE_MODEL = os.getenv("YANTRA_TRANSLATE_MODEL", OPENAI_MODEL)
33
34 # how many sales nudges to present per conversation before backing off
35 SALES_NUDGE_LIMIT = int(os.getenv("YANTRA_SALES_NUDGE_LIMIT", "2"))
36
37 SUPPORT_PHONE = os.getenv("YANTRALIVE_SUPPORT_PHONE", "+91-9876543210")
38 SUPPORT_EMAIL = os.getenv("YANTRALIVE_SUPPORT_EMAIL", "support@yantralive.com")
39
40 # STT config
41 YANTRA_STT_MAX_BYTES = int(os.getenv("YANTRA_STT_MAX_BYTES", str(12 * 1024 * 1024)))
42 YANTRA_STT_TMP_DIR = os.getenv("YANTRA_STT_TMP_DIR", None) # optional custom tmp dir
43
44 if not OPENAI_API_KEY:
45     raise RuntimeError("Missing OPENAI_API_KEY in .env")
46
47 openai_client = OpenAI(api_key=OPENAI_API_KEY)

```

```

48
49 # =====
50 # FASTAPI
51 # =====
52 app = FastAPI(
53     title="YantraLive RAG Chatbot (OpenAI)",
54     version="1.5",
55 )
56
57 app.add_middleware(
58     CORSMiddleware,
59     allow_origins=["*"],
60     allow_methods=["*"],
61     allow_headers=["*"],
62 )
63
64 # =====
65 # Serve brochures (static) + preview wrapper endpoints
66 # =====
67 BROCHURE_DIR = os.path.join("data", "brochures")
68 os.makedirs(BROCHURE_DIR, exist_ok=True)
69 app.mount("/static_brochures", StaticFiles(directory=BROCHURE_DIR), name="static_broc
70
71
72 ✓ def _build_brochure_map():
73     m = {}
74     if not os.path.isdir(BROCHURE_DIR):
75         return m
76     for fname in os.listdir(BROCHURE_DIR):
77         if not fname.lower().endswith(".pdf"):
78             continue
79         name_no_ext = os.path.splitext(fname)[0]
80         key = re.sub(r"^[a-z0-9]", "", name_no_ext.lower())
81         m[key] = fname
82     return m
83
84
85 BROCHURE_MAP = _build_brochure_map()
86
87
88 @app.get("/api/brochures/list")
89 def list_brochures():
90     return JSONResponse(BROCHURE_MAP)
91
92
93 @app.get("/brochures/view/{filename}", response_class=HTMLResponse)
94 ✓ def brochure_view(filename: str):
95     safe = os.path.basename(filename)
96     path = os.path.join(BROCHURE_DIR, safe)
97     if not os.path.isfile(path):
98         raise HTTPException(status_code=404, detail="Brochure not found")
99     raw_url = f"/brochures/raw/{safe}"
100     html = f"""<!doctype html>
101     <html>
102     <head>

```



main ▾

yantra-chatbot / yantra-backend / main.py

↑ Top

Code

Blame



Raw



```

107 .html,body {{ height:100%; margin:0; background:#f7f7f7; }}
108 .topbar {{ padding:10px; background:#fff; border-bottom:1px solid #eee; display:f
109 .open-btn {{ padding:6px 10px; border-radius:6px; border:1px solid #ccc; backgrou
110 .iframe-wrap {{ height: calc(100% - 52px); }}

```

```

111         iframe {{ width:100%; height:100%; border:none; }}
112         .fallback {{ padding:20px; font-family:system-ui, -apple-system, "Segoe UI", Robo
113     </style>
114 </head>
115 <body>
116     <div class="topbar">
117         <strong>{safe}</strong>
118         <a class="open-btn" href="{raw_url}" target="_blank" rel="noopener noreferrer">Op
119         <a class="open-btn" href="{raw_url}" download>Download</a>
120     </div>
121     <div class="iframe-wrap" role="document">
122         <iframe src="{raw_url}#toolbar=1" title="brochure">
123             <div class="fallback">
124                 Your browser couldn't display the PDF inline. <a href="{raw_url}" target="_bl
125             </div>
126         </iframe>
127     </div>
128 </body>
129 </html>""
130     return HTMLResponse(content=html, status_code=200)
131
132
133 @app.get("/brochures/raw/{filename}")
134 ✓ def brochure_raw(filename: str):
135     safe = os.path.basename(filename)
136     path = os.path.join(BROCHURE_DIR, safe)
137     if not os.path.isfile(path):
138         raise HTTPException(status_code=404, detail="Brochure not found")
139     headers = {
140         "Content-Disposition": f'inline; filename="{safe}"',
141         "Accept-Ranges": "bytes",
142         "X-Content-Type-Options": "nosniff",
143         "Cache-Control": "public, max-age=0, must-revalidate",
144     }
145     return FileResponse(path, media_type="application/pdf", headers=headers)
146
147
148 # =====
149 # CHROMA
150 # =====
151 chroma_client = chromadb.Client()
152
153
154 ✓ def create_or_get_collection(name: str):
155     try:
156         return chroma_client.create_collection(name=name)
157     except Exception:
158         return chroma_client.get_collection(name=name)
159
160
161 end_customer_collection = create_or_get_collection("yantra_end_customer")
162 spare_parts_collection = create_or_get_collection("yantra_spare_parts")
163 dealer_collection = create_or_get_collection("yantra_dealers")
164
165
166 # =====
167 # Pydantic MODELS
168 # =====
169 class ChatMessage(BaseModel):
170     role: Literal["user", "assistant"]
171     content: str
172
173

```

```

174     class ChatRequest(BaseModel):
175         messages: List[ChatMessage]
176
177
178     class ChatResponse(BaseModel):
179         answer: str
180         used_context: List[str]
181         from_fallback: bool = False
182         brochure_url: Optional[str] = None
183         followup_meta: Optional[Dict[str, Any]] = None
184
185
186     # =====
187     # OPENAI EMBEDDINGS + RETRY
188     # =====
189     EMBED_BATCH_SIZE = 64
190     EMBED_BATCH_SLEEP_SECONDS = 1.0
191
192
193     def _openai_embed_with_retry(
194         texts: List[str],
195         model: str,
196         label: str = "",
197         max_retries: int = 5,
198     ) -> List[List[float]]:
199         for attempt in range(max_retries):
200             try:
201                 resp = openai_client.embeddings.create(model=model, input=texts)
202                 data = getattr(resp, "data", resp.get("data") if isinstance(resp, dict) else None)
203                 if data is None:
204                     data = resp["data"]
205                 embeddings = []
206                 for d in data:
207                     if isinstance(d, dict):
208                         embeddings.append(d.get("embedding"))
209                     else:
210                         embeddings.append(getattr(d, "embedding", None))
211                 return embeddings
212             except Exception as e:
213                 msg = str(e).lower()
214                 if "rate limit" in msg or "429" in msg:
215                     wait = 5 * (attempt + 1)
216                     print(
217                         f"[OPENAI] Rate limited while embedding {label} "
218                         f"(attempt {attempt + 1}/{max_retries}). Sleeping {wait}s."
219                     )
220                     time.sleep(wait)
221                     continue
222
223                 print(f"[OPENAI] Non-rate-limit error while embedding {label}: {e}")
224                 raise
225
226             raise RuntimeError(f"OpenAI embed retries exceeded for {label}")
227
228
229     def embed_documents(texts: List[str]) -> List[List[float]]:
230         return _openai_embed_with_retry(
231             texts=texts,
232             model=OPENAI_EMBED_MODEL,
233             label=f"documents batch (size={len(texts)}",
234         )
235
236

```

```

237  ✓ def embed_query(text: str) -> List[float]:
238      embeddings = _openai_embed_with_retry(
239          texts=[text],
240          model=OPENAI_EMBED_MODEL,
241          label="user query",
242      )
243      return embeddings[0]
244
245
246  # =====
247  # LOAD CSV + INDEX
248  # =====
249  DATA_DIR = "data"
250  END_CUSTOMER_FILE = os.path.join(DATA_DIR, "end_customer.csv")
251  SPARE_PARTS_FILE = os.path.join(DATA_DIR, "spare_parts.csv")
252  DEALERS_FILE = os.path.join(DATA_DIR, "dealers.csv")
253
254
255  ✓ def load_and_index_one(path: str, collection, tag: str):
256      if not os.path.exists(path):
257          print(f"[INFO] Dataset not found for {tag}: {path} (skipping)")
258          return
259
260      # use robust CSV loading
261      try:
262          df = pd.read_csv(path, encoding="utf-8", dtype=str).fillna("")
263      except Exception:
264          # fallback to default pandas guessing if encoding utf-8 fails
265          df = pd.read_csv(path, dtype=str).fillna("")
266
267      if df.empty:
268          print(f"[WARN] Dataset {tag} is empty: {path}")
269          return
270
271      documents: List[str] = []
272      ids: List[str] = []
273
274      for i, row in df.iterrows():
275          row_text = " | ".join([f"{col}: {row[col]}" for col in df.columns])
276          documents.append(f"[{tag}] {row_text}")
277          ids.append(f"{tag.lower()}_{row_i}")
278
279      total_docs = len(documents)
280      print(f"[INDEX] Starting indexing for {tag}: {total_docs} rows")
281
282      try:
283          for start in range(0, total_docs, EMBED_BATCH_SIZE):
284              end = min(start + EMBED_BATCH_SIZE, total_docs)
285              batch_docs = documents[start:end]
286              batch_ids = ids[start:end]
287
288              try:
289                  batch_vectors = embed_documents(batch_docs)
290              except Exception as batch_err:
291                  print(
292                      f"[WARN] Failed to embed batch {start}:{end} for {tag}: {batch_err}"
293                  )
294                  continue
295
296              collection.add(
297                  ids=batch_ids,
298                  documents=batch_docs,
299                  embeddings=batch_vectors,

```

```

300         )
301         print(f"[INDEX] Indexed rows {start} to {end - 1} for {tag}")
302         time.sleep(EMBED_BATCH_SLEEP_SECONDS)
303
304         print(
305             f"[INDEX] Finished indexing {total_docs} rows for {tag} using OpenAI embed
306         )
307     except Exception as e:
308         print(f"[WARN] Failed to embed/index dataset {tag} with OpenAI: {e}")
309         print("[WARN] Starting server without this vector index; chat may fallback.")
310
311
312     def load_all_datasets():
313         load_and_index_one(END_CUSTOMER_FILE, end_customer_collection, "END_CUSTOMER")
314         time.sleep(2)
315         load_and_index_one(SPARE_PARTS_FILE, spare_parts_collection, "SPARE_PARTS")
316         time.sleep(2)
317         load_and_index_one(DEALERS_FILE, dealer_collection, "DEALERS")
318
319
320     try:
321         # IMPORTANT: Keep startup resilient – indexing runs synchronously here but errors
322         load_all_datasets()
323     except Exception as e:
324         print(f"[WARN] Dataset indexing failed on startup: {e}")
325
326
327     # =====
328     # FALLBACK MESSAGE / DEFAULTS
329     # =====
330     DEFAULT_WARRANTY_TEXT = (
331         "Standard warranty: 1-year warranty covering the piston, the piston body, the cyl
332         "Warranty is conditional on regular maintenance (maintenance mandatory to claim w
333         "Parts INCLUDED under warranty: piston, piston body, front head.\n"
334         "Parts EXCLUDED from warranty: back head, seal, control valve, bushes, chisels, e
335     )
336     DEFAULT_GST_TEXT = "GST: 18% applies on breakers/parts unless a CONTEXT row explicitl
337
338     def fallback() -> str:
339         return (
340             "I couldn't find this information in the latest Yantralive dataset.\n\n"
341             f"Please contact human support:\n"
342             f"☎ {SUPPORT_PHONE}\n"
343             f"✉ {SUPPORT_EMAIL}"
344         )
345
346
347     # =====
348     # NORMALIZER
349     # =====
350     def auto_normalize_sb(text: str) -> str:
351         if not text:
352             return text
353         pattern = re.compile(r"\b(SB)(?:[-\s]*)(\d*)\b", re.IGNORECASE)
354         def repl(m: re.Match) -> str:
355             digits = m.group(2) or ""
356             return "VJ" + digits
357         return pattern.sub(repl, text)
358
359
360     # =====
361     # Translation helper (to English)
362     # =====

```

```

363  ✓ def _translate_to_english(text: str) -> str:
364      """
365      Translate the input text to English using the OpenAI chat completion endpoint.
366      Returns empty string on failure.
367      """
368      if not text:
369          return ""
370      try:
371          # Keep translation prompt short and deterministic
372          sys_prompt = "You are a translation helper. Translate the user's message to f
373          user_prompt = f"Translate to English (only the translation, no extra text):\n
374          resp = openai_client.chat.completions.create(
375              model=TRANSLATE_MODEL,
376              messages=[
377                  {"role": "system", "content": sys_prompt},
378                  {"role": "user", "content": user_prompt},
379              ],
380              temperature=0.0,
381              max_tokens=256,
382          )
383          choices = getattr(resp, "choices", resp.get("choices")) if isinstance(resp, di
384          if not choices:
385              return ""
386          choice = choices[0] if isinstance(choices, (list, tuple)) else choices
387          # extract content safely
388          msg = None
389          if isinstance(choice, dict):
390              msg = choice.get("message") or choice.get("text") or choice.get("content"
391          else:
392              msg = getattr(choice, "message", None) or getattr(choice, "text", None) o
393          if isinstance(msg, dict):
394              return (msg.get("content") or msg.get("text") or "").strip()
395          if isinstance(msg, str):
396              return msg.strip()
397          return ""
398      except Exception as e:
399          print(f"[TRANSLATE] translation failed: {e}")
400          return ""
401
402
403      # =====
404      # RAG SYSTEM PROMPT & GENERATION (kept as requested)
405      # =====
406      GROQ_MODEL_ID = "llama-3.3-70b-versatile" # parity-only tag
407
408  ✓ def _extract_choice_message(choice: Any) -> str:
409      msg = None
410      if isinstance(choice, dict):
411          msg = choice.get("message") or choice.get("text") or choice.get("content")
412      else:
413          msg = getattr(choice, "message", None) or getattr(choice, "text", None) or ge
414      if isinstance(msg, dict):
415          return msg.get("content") or msg.get("text") or ""
416      if isinstance(msg, str):
417          return msg
418      return getattr(msg, "content", "") if msg is not None else ""
419
420
421  ✓ def generate_with_groq(context: str, user_question: str, user_ton: Optional[int] = No
422      prefix_rule = (
423          "IMPORTANT - ANSWERING GUIDELINES (apply these before any other instruction):
424          "- Treat SB-* mentions internally as VJ-* (do this silently). Never mention o
425          "- Do NOT include any provenance or extraction notes (e.g., 'extracted from .

```

```

426         "- If the user mentions ONLY a machine model (e.g., 'Hyundai R30') return ALL
427         " Provide full rows / all dataset columns and keep language natural and help
428         "- If the user mentions ONLY a breaker model (e.g., 'VJ20 HD') return ALL det
429         " EXCLUDING the 'compatible machines' section initially. After listing break
430         "- If there are multiple compatible breakers, list them all. Do not add a 'th
431         "- Keep responses concise, human-friendly, and start direct answers with a sh
432         "- Do not reveal internal normalizations or synonyms. If user typed SB*, simp
433         "- Avoid extra filler lines. Answer to the point.\n\n"
434         "TONNAGE HANDLING RULES (use these to match ranges):\n"
435         "- When the user provides a numeric tonnage value (e.g., '8 ton', '9 t', '10'
436         "- If a CONTEXT row contains a range like '7-10 Ton' then integer values 7, 8
437         "- Accept range formats like '7-10 Ton', '7 to 10 Ton', '7-10 Ton', '7 - 10
438         "- If the user requests a single ton value, prefer breakers where the CONTEXT
439         "- If the CONTEXT doesn't have explicit ranges covering the value, reply EXAC
440         "WARRANTY & STANDARD TERMS (default policy to use when CONTEXT lacks explicit
441         f"- {DEFAULT_WARRANTY_TEXT.replace(chr(10), chr(10) + '- ')}\n"
442         f"- {DEFAULT_GST_TEXT}\n"
443         "- Payment terms (default): 100% advance required before dispatch.\n"
444         "- Dispatch ETA (default): within 24 hours of full payment received, unless C
445         "- Standard package inclusions (default): two chisels, one empty gas cylinder
446         "- Transport (default): shipped on two-pay basis via VRL by default, or trans
447         "- IMPORTANT: Use these warranty/terms only when the CONTEXT does not present
448         "SALES FOLLOW-UP RULES (when user asks about a specific breaker/model):\n"
449         "- After answering facts about a specific breaker/model, present a short poli
450         "- Suggested follow-ups: brochure, connect to dealer/contact for quote, get p
451         "- Respect user's negative response. If the user replies 'no' or indicates di
452         "- Do not be aggressive; keep follow-ups helpful and concise.\n\n"
453         "STRICT RAG & RANGE MATCHING RULES:\n"
454         "- You MUST use ONLY the facts from the CONTEXT rows and the explicit USER_TO
455         "- If the answer is not supported by the CONTEXT and not derivable from USER_
456         "- Do NOT invent models, breakers, dealers, or parts. Do NOT use outside know
457     )
458
459     system_prompt = (
460         "You are a strict RAG assistant for YantraLive END-CUSTOMER, SPARE_PARTS, and
461         "\n"
462         "GENERAL RULES:\n"
463         "- You MUST use ONLY the facts from the CONTEXT.\n"
464         "- If the answer is not clearly present in the CONTEXT, reply EXACTLY: UNSURE
465         "- Do NOT guess. Do NOT use outside knowledge except the explicit warranty &
466         "- Keep answers concise, factual, and formatted cleanly.\n"
467         "- Respect the dataset tags [END_CUSTOMER], [SPARE_PARTS], [DEALERS] when rea
468         "\n"
469         "COMPATIBILITY QUESTIONS:\n"
470         "- Scan EVERY row in the CONTEXT.\n"
471         "- Identify all rows where the machine brand and/or machine model match the u
472         "- For tonnage queries: match numeric ton values against any capacity/tonnage
473         "- Extract all DISTINCT compatible breaker models / SKUs and present them in
474         "\n"
475         "OUTPUT FORMATS:\n"
476         "Compatibility:\n"
477         "***Compatible Breakers:**\n"
478         "- <Breaker Model / SKU> - <key facts from dataset only>\n"
479         "- <Breaker Model / SKU> - <key facts from dataset only>\n"
480         "\n"
481         "Comparison:\n"
482         "Start: 'Here is a comparison between <X> and <Y>:' then a markdown table:\n"
483         "| Feature | Option 1 | Option 2 |\n"
484         "|-----|-----|-----|\n"
485         "\n"
486         "Specific parameter questions:\n"
487         "- Extract the exact requested numeric or textual values. If multiple rows gi
488         "\n"

```



```

489         "SUBJECTIVE / BEST-OPTION QUESTIONS:\n"
490         "- List all relevant options as bullets with key specs, then choose ONE option"
491         "\n"
492         "REMEMBER:\n"
493         "- Never invent breakers, dealers, or spare parts that are not present in the"
494         "- If the machine, breaker, part, or dealer mentioned is not present at all,"
495     )
496
497     full_system_prompt = prefix_rule + system_prompt
498
499     user_content = f"""
500     CONTEXT (rows from YantraLive datasets):
501     {context}
502
503     USER QUESTION:
504     {user_question}
505     """
506
507     if user_ton is not None:
508         user_content += f"\nUSER_TON: {user_ton}\n"
509
510     try:
511         messages = [
512             {"role": "system", "content": full_system_prompt},
513             {"role": "user", "content": user_content},
514         ]
515
516         resp = openai_client.chat.completions.create(
517             model=OPENAI_MODEL,
518             messages=messages,
519             temperature=0.0,
520             max_tokens=1500,
521         )
522
523         choices = getattr(resp, "choices", resp.get("choices")) if isinstance(resp, dict) else None
524         if choices is None:
525             choices = resp["choices"]
526         choice = choices[0] if isinstance(choices, (list, tuple)) and len(choices) > 0 else None
527         answer = _extract_choice_message(choice)
528         return answer
529     except Exception as e:
530         print(f"[OPENAI ERROR] {e}")
531         return None
532
533     # =====
534     # Helper deterministic extraction utilities
535     # =====
536     def _find_vj_tokens_in_text(text: str) -> Set[str]:
537         if not text:
538             return set()
539
540         tokens = set(re.findall(r"\b(vj)[\s-]*?(\d{1,3})(?:\s*(hd))?\b", text, flags=re.IGNORECASE))
541         out = set()
542         for t in tokens:
543             joined = "".join([p for p in t if p])
544             normalized = re.sub(r"[^a-z0-9]", "", joined.lower())
545             out.add(normalized)
546         return out
547
548     def _extract_field_values_from_row(row_text: str, field_names: List[str]) -> dict:
549         pairs = re.findall(r"([A-Za-z0-9_/()%-+):\s*([^\n]+)", row_text)
550         d = {}
551         for (k, v) in pairs:

```

```

552         d[k.strip().lower()] = v.strip()
553     res = {}
554     for fn in field_names:
555         res[fn] = d.get(fn.lower())
556     return res
557
558
559     def _collect_all_compatible_machines_or_breakers(query_token: str, docs: List[str]) -
560         """
561         Keep existing collector for compatibility detection – returns set of tokens and s
562         """
563         found: Set[str] = set()
564         source_rows: List[str] = []
565         qlow = query_token.lower()
566         for d in docs:
567             low = d.lower()
568             if qlow in low:
569                 source_rows.append(d)
570                 pairs = re.findall(r"([A-Za-z0-9 _/()%~+]:\s*([^\n]+)", d)
571                 for (k, v) in pairs:
572                     k_clean = k.strip().lower()
573                     if "compatible" in k_clean:
574                         tokens = re.split(r"[;,/\|]+", v)
575                         for t in tokens:
576                             t = t.strip()
577                             if not t:
578                                 continue
579                             vj_matches = re.findall(r"\b(vj)[\s-]*?(\d{1,3})(?:\s*(hd))?"
580                             if vj_matches:
581                                 for m in vj_matches:
582                                     joined = "".join([p for p in m if p])
583                                     normalized = re.sub(r"^[a-z0-9]", "", joined.lower())
584                                     found.add(normalized)
585                             else:
586                                 norm = re.sub(r"^[a-z0-9 ]", "", t.lower())
587                                 if norm:
588                                     found.add(norm)
589                             vjset = _find_vj_tokens_in_text(d)
590                             found.update(vjset)
591         return found, source_rows
592
593
594     def _is_simple_negative_reply(text: str) -> bool:
595         if not text:
596             return False
597         text = text.strip().lower()
598         negatives = {"no", "nope", "nah", "not now", "dont want", "don't want", "no thank
599         if text in negatives:
600             return True
601         if re.fullmatch(r"no[.!]??", text):
602             return True
603         # also accept some common hindi/kannada negatives (basic)
604         if text in {"नहीं", "नो", "अच्छे"}:
605             return True
606         return False
607
608
609     def _count_prior_followups(messages: List[dict]) -> int:
610         count = 0
611         for m in messages:
612             if m.get("role") == "assistant" and "Would you like any of the following" in
613                 count += 1
614         return count

```

```

615
616
617 ✓ def _parse_followup_comment(text: str) -> Optional[Dict[str, Any]]:
618     """
619     Parse an HTML comment tag we add to answers, e.g.:
620     <!--MODEL_CTX:vj20hd|OPTIONS:brochure,connect,price,specs-->
621     Returns dict {model: 'vj20hd', options: ['brochure', 'connect', ...]} or None.
622     """
623     if not text:
624         return None
625     m = re.search(r"<!--\s*MODEL_CTX:([a-z0-9]+)\|OPTIONS:([a-z0-9,]+)\s*-->", text,
626     if not m:
627         return None
628     model = m.group(1).lower()
629     opts = [o.strip() for o in m.group(2).split(",") if o.strip()]
630     return {"model": model, "options": opts}
631
632
633 ✓ def _find_price_for_model(key_norm: str, docs: List[str]) -> Optional[str]:
634     # look for common price-related fields in the docs that match the model
635     price_keys = ["price", "mrp", "price (in", "price incl", "price including", "price
636     found_prices: Set[str] = set()
637     for d in docs:
638         if key_norm not in d.lower():
639             continue
640         pairs = re.findall(r"([A-Za-z0-9 _/()%-]+):\s*([^\n]+)", d)
641         for (k, v) in pairs:
642             k_clean = k.strip().lower()
643             for pk in price_keys:
644                 if pk in k_clean:
645                     found_prices.add(v.strip())
646     if found_prices:
647         return "; ".join(sorted(found_prices))
648     return None
649
650
651 ✓ def _query_collection(coll, label: str, query_vec, n_results=50):
652     """
653     Unified query wrapper. Increased n_results for broader results (so "all compatibl
654     """
655     try:
656         result = coll.query(
657             query_embeddings=query_vec,
658             n_results=n_results,
659         )
660         # return list of documents (flatten)
661         docs = []
662         try:
663             # new style returns dict with 'documents'
664             docs_list = result.get("documents", [])
665             # documents may be list of lists per query
666             if docs_list and isinstance(docs_list[0], list):
667                 docs = docs_list[0]
668             else:
669                 docs = docs_list or []
670         except Exception:
671             # fallback defensive handling
672             if isinstance(result, dict) and "documents" in result:
673                 docs = result["documents"][0] if result["documents"] else []
674     except Exception as e:
675         print(f"[ERROR] Failed to query {label} collection: {e}")
676         return []
677     return docs

```

```

678
679
680 # =====
681 # New helpers to produce the requested output formatting for breaker queries
682 # =====
683 ✓ def _extract_pairs_dict_from_row(row_text: str) -> Dict[str, str]:
684     """
685     Extract key:value pairs from a row string and return a dict with normalized (lower
686     """
687     pairs = re.findall(r"([A-Za-z0-9 _/()%~+]:\s*([^\n]+)", row_text)
688     d: Dict[str, str] = {}
689     for (k, v) in pairs:
690         k_clean = k.strip()
691         v_clean = v.strip()
692         if k_clean:
693             d[k_clean.lower()] = v_clean
694     return d
695
696
697 ✓ def _merge_rows_to_field_map(rows: List[str]) -> Dict[str, str]:
698     """
699     Given multiple dataset row-strings that mention the same breaker, merge fields in
700     Preference: keep the first non-empty value seen for a key.
701     """
702     merged: Dict[str, str] = {}
703     for r in rows:
704         d = _extract_pairs_dict_from_row(r)
705         for k, v in d.items():
706             if not v:
707                 continue
708             if k not in merged or not merged[k]:
709                 merged[k] = v
710     return merged
711
712
713 ✓ def _extract_machine_brand_model_from_row_dict(row_dict: Dict[str, str]) -> Optional[
714     """
715     Given a row dict (lowercase keys), extract a 'Brand - Model' string if possible.
716     Looks for common keys like 'machine brand' and 'machine model' or variants.
717     """
718     brand_keys = ["machine brand", "brand", "maker"]
719     model_keys = ["machine model", "model", "machine", "machine name"]
720     brand = None
721     model = None
722     for bk in brand_keys:
723         if bk in row_dict and row_dict[bk].strip():
724             brand = row_dict[bk].strip()
725             break
726     for mk in model_keys:
727         if mk in row_dict and row_dict[mk].strip():
728             model = row_dict[mk].strip()
729             break
730     if brand and model:
731         # sanitize whitespace
732         return f"{brand} - {model}"
733     # if brand present only, return brand
734     if brand and not model:
735         return brand
736     # if model present only, return model
737     if model and not brand:
738         return model
739     return None
740

```



```

804         for t in tokens:
805             cleaned = _clean_token(t)
806             if not cleaned:
807                 continue
808             if " " in cleaned:
809                 parts = cleaned.split()
810                 brand = parts[0].strip()
811                 model = " ".join(parts[1:]).strip()
812                 if brand and model:
813                     machines_set.add(f"{brand} - {model}")
814                 continue
815             machines_set.add(cleaned)
816
817     # 3) Normalize and return sorted list for deterministic order
818     final = sorted({re.sub(r"\s+", " ", m).strip() for m in machines_set})
819     return final
820
821
822     # =====
823     # ROUTES
824     # =====
825     @app.get("/api/health")
826     def health():
827         return {"status": "ok"}
828
829
830     # Helper for greetings & time-based salutation
831     ✓ def _is_greeting(text: str) -> bool:
832         if not text:
833             return False
834         t = text.strip().lower()
835         greetings = {
836             "hi", "hello", "hey", "good morning", "good afternoon", "good evening",
837             "gm", "hello!", "hi!", "hey!", "thanks", "thank you", "thankyou",
838             # some localized greetings (basic)
839             "नमस्ते", "नमस्कार", "हैलो", "हाय", "ಹೇಲೋ", "ஹலோ"
840         }
841         # allow startswith checks for e.g. "good morning everyone"
842         for g in greetings:
843             if t == g or t.startswith(g + " ") or t.startswith(g + "!") or t.startswith(g
844                 return True
845         return False
846
847
848     ✓ def _time_based_greeting() -> str:
849         # Asia/Kolkata timezone per project settings
850         try:
851             now = datetime.now(ZoneInfo("Asia/Kolkata"))
852         except Exception:
853             now = datetime.utcnow()
854         hour = now.hour
855         if 5 <= hour < 12:
856             return "Good morning"
857         if 12 <= hour < 17:
858             return "Good afternoon"
859         if 17 <= hour < 22:
860             return "Good evening"
861         return "Hello"
862
863
864     ✓ def _extract_ton_from_text(text: str) -> Optional[int]:
865         if not text:
866             return None

```

```

867 # robust patterns: single "8 ton", "8t", "8 T", numbers with 'ton(s)', or plain n
868 m = re.search(r"\b(\d{1,3})\s*(?:t(?:on)?s?|tonnes?|tons?)\b", text, re.IGNORECAS
869 if m:
870     try:
871         return int(m.group(1))
872     except:
873         return None
874 # plain "8T" or "8T" style with optional spaces
875 m2 = re.search(r"\b(\d{1,3})\s*[tT]\b", text)
876 if m2:
877     try:
878         return int(m2.group(1))
879     except:
880         return None
881 # sometimes user may just type a number with 'ton' implied but we avoid over-trig
882 return None
883
884
885 @app.post("/api/stt")
886 ✓ async def stt_endpoint(file: UploadFile = File(...), lang: Optional[str] = "en"):
887     """
888     POST /api/stt?lang=<en|hi|kn>
889     - file: form-data upload (UploadFile)
890     - returns JSON: { "transcript": "..."}
891     """
892     # Basic checks
893     if not file:
894         raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="No file
895
896     # sanitize filename
897     filename = os.path.basename(file.filename or "upload")
898     # stream size guard: read header chunk to ensure content-type is audio and not la
899     # Note: UploadFile.file is a SpooledTemporaryFile – we'll stream to a temp file a
900     tmp_dir = YANTRA_STT_TMP_DIR or None
901     tmp_file = None
902     try:
903         with tempfile.NamedTemporaryFile(delete=False, suffix=os.path.splitext(filena
904             tmp_file = tf.name
905             total = 0
906             # stream write
907             while True:
908                 chunk = await file.read(1024 * 64)
909                 if not chunk:
910                     break
911                 total += len(chunk)
912                 if total > YANTRA_STT_MAX_BYTES:
913                     # cleanup and abort
914                     tf.close()
915                     try:
916                         os.unlink(tmp_file)
917                     except Exception:
918                         pass
919                     raise HTTPException(status_code=413, detail=f"Uploaded file excee
920                 tf.write(chunk)
921             tf.flush()
922
923     # call OpenAI transcription (whisper-1)
924     # We let the model auto-detect format; we forward the 'language' parameter wh
925     transcript_text = None
926     try:
927         with open(tmp_file, "rb") as fh:
928             # The OpenAI SDK wrapper used here: openai_client.audio.transcription
929             # using model "whisper-1" which is commonly available

```

```

930         kwargs = {"file": fh, "model": "whisper-1"}
931         if lang:
932             kwargs["language"] = lang
933         resp = openai_client.audio.transcriptions.create(**kwargs)
934         # resp may be object-like or dict-like; read 'text' or 'transcript'
935         transcript_text = None
936         if isinstance(resp, dict):
937             transcript_text = resp.get("text") or resp.get("transcript") or ""
938         else:
939             # attempt getattr
940             transcript_text = getattr(resp, "text", None) or getattr(resp, "transcript", None)
941         except Exception as e:
942             print(f"[STT] OpenAI transcription error: {e}")
943             raise HTTPException(status_code=500, detail="Transcription failed: " + str(e))
944
945         if transcript_text is None:
946             transcript_text = ""
947
948         return JSONResponse({"transcript": transcript_text})
949     finally:
950         try:
951             if tmp_file and os.path.exists(tmp_file):
952                 os.unlink(tmp_file)
953         except Exception:
954             pass
955
956
957     # =====
958     # /api/chat implementation (updated for multilingual keyword checks)
959     # =====
960     @app.post("/api/chat", response_model=ChatResponse)
961     ✓ def chat(req: ChatRequest, request: Request):
962         if not req.messages:
963             raise HTTPException(status_code=400, detail="No messages provided")
964
965         user_msg_raw = req.messages[-1].content
966         print(f"[CHAT] Original User message: {user_msg_raw!r}")
967
968         normalized_user_msg = auto_normalize_sb(user_msg_raw)
969         if normalized_user_msg != user_msg_raw:
970             print(f"[CHAT] Normalized User message (SB->VJ): {normalized_user_msg!r}")
971         else:
972             print(f"[CHAT] Normalized User message: {normalized_user_msg!r}")
973
974         last_user = normalized_user_msg.strip()
975
976         # Create a translated-to-English variant for keyword checks
977         low = last_user.lower().strip()
978         low_en = ""
979         try:
980             low_en = _translate_to_english(last_user) or ""
981             low_en = low_en.lower().strip()
982         except Exception as e:
983             print(f"[CHAT] Translation attempt failed: {e}")
984             low_en = ""
985
986         # Combine both original and translated text for keyword detection
987         low_both = " ".join(filter(None, [low, low_en]))
988
989         # Immediately handle greetings (do not proceed to RAG)
990         # Use either the original language or the translated English to detect greetings
991         if _is_greeting(low) or (low_en and _is_greeting(low_en)):
992             sal = _time_based_greeting()
993             # preserve short polite form for thanks

```



```

993         if any(tok in low_both for tok in ("thank", "thanks", "thankyou", "thanks!",
994             return ChatResponse(answer="You're welcome! Anything else I can help with
995             return ChatResponse(answer=f"{sal}! How can I help you today?", used_context=
996
997     msgs_for_state = [m.dict() for m in req.messages]
998     prior_followups = _count_prior_followups(msgs_for_state)
999     print(f"[CHAT] Prior followups presented so far: {prior_followups}")
1000
1001     # If previous assistant offered followups and included the hidden MODEL_CTX comme
1002     if len(req.messages) >= 2 and req.messages[-2].role == "assistant":
1003         prev_assistant_text = req.messages[-2].content or ""
1004         followup_meta = _parse_followup_comment(prev_assistant_text)
1005         # Simple negative handling unchanged but multilingual-aware
1006         if "Would you like any of the following for this model?" in prev_assistant_te
1007             if _is_simple_negative_reply(low_both):
1008                 reply_text = "No problem – I won't suggest that again right now. Anyt
1009                 return ChatResponse(answer=reply_text, used_context=[], from_fallback
1010
1011         if followup_meta:
1012             # followup flow: prefer explicit option keywords, then handle simple affi
1013             model_key = followup_meta.get("model")
1014             options = followup_meta.get("options", [])
1015
1016             # Standard option keywords mapping (include some localized tokens)
1017             option_keywords = {
1018                 "brochure": ["brochure", "pdf", "open brochure", "brochar", "ब्रोशर",
1019                 "connect": ["connect", "dealer", "contact", "quote", "connect me", "d
1020                 "price": ["price", "quote", "proforma", "cost", "how much", "dhaam",
1021                 "specs": ["specs", "warranty", "specification", "details", "warranty
1022             }
1023
1024             # If user provided a clear option request (use low_both)
1025             for opt, keywords in option_keywords.items():
1026                 if any(k in low_both for k in keywords) and opt in options:
1027                     # handle same as previous code paths
1028                     if opt == "brochure":
1029                         fname = BROCHURE_MAP.get(model_key) or BROCHURE_MAP.get(re.su
1030                         if fname:
1031                             base = str(request.base_url).rstrip("/")
1032                             brochure_url = f"{base}/brochures/view/{fname}"
1033                             reply = f"I've opened the brochure for {model_key.upper()}
1034                             return ChatResponse(answer=reply, used_context=[], from_f
1035                             return ChatResponse(answer="I couldn't find a brochure for th
1036
1037                     if opt == "connect":
1038                         reply = (
1039                             f"I can connect you with a dealer. Please share your pref
1040                             f"Or I can share our support contact: 📞 {SUPPORT_PHONE}
1041                         )
1042                         return ChatResponse(answer=reply, used_context=[], from_fallb
1043
1044                     if opt == "price":
1045                         try:
1046                             query_vec = embed_query(model_key)
1047                         except Exception as e:
1048                             print(f"[ERROR] Failed to embed model query for price: {e
1049                             return ChatResponse(answer=fallback(), used_context=[], f
1050
1051                     docs: List[str] = []
1052                     docs.extend(_query_collection(end_customer_collection, "END_C
1053                     docs.extend(_query_collection(spare_parts_collection, "SPARE_
1054                     docs.extend(_query_collection(dealer_collection, "DEALERS", q
1055

```

```

1056         price = _find_price_for_model(model_key, docs)
1057         if price:
1058             reply = f"Here is the price information I found for {model_key}"
1059             return ChatResponse(answer=reply, used_context=docs or [])
1060         else:
1061             reply = (
1062                 f"I couldn't find a direct price record for {model_key}. "
1063                 f"I can prepare a quote/proforma for you – please share your requirements."
1064             )
1065             return ChatResponse(answer=reply, used_context=docs or [])
1066
1067     if opt == "specs":
1068         # call RAG generator for warranty/specs specifically
1069         try:
1070             query_vec = embed_query(model_key)
1071         except Exception as e:
1072             print(f"[ERROR] Failed to embed model query for specs: {e}")
1073             return ChatResponse(answer=fallback(), used_context=[], from_fallback=True)
1074
1075         docs: List[str] = []
1076         docs.extend(_query_collection(end_customer_collection, "END_CUSTOMER", query_vec))
1077         docs.extend(_query_collection(spare_parts_collection, "SPARE_PARTS", query_vec))
1078         docs.extend(_query_collection(dealer_collection, "DEALERS", query_vec))
1079
1080         raw = generate_with_groq(context="\n\n---\n\n".join(docs), user_query=query, temperature=0.1)
1081         if raw is None or "UNSURE_FROM_DATA" in raw:
1082             return ChatResponse(answer=DEFAULT_WARRANTY_TEXT, used_context=docs or [])
1083         return ChatResponse(answer=raw.strip(), used_context=docs or [])
1084
1085     # If user replied with a simple affirmative (yes/ok/sure) – ask which option they want
1086     affirmatives = {"yes", "y", "yeah", "yep", "sure", "ok", "okay", "please", "yes please"}
1087     if low_both in affirmatives or any(low_both.strip().startswith(a) for a in affirmatives):
1088         return ChatResponse(answer="Sure – which option would you like? You can choose from the options below.", used_context=docs or [])
1089
1090     # If user replied with unrelated short replies (single word stopwords), ask for clarification
1091     short_stopwords = {"hi", "hello", "hey", "yes", "no", "ok", "okay", "thank you", "thanks"}
1092     if low_both in short_stopwords:
1093         return ChatResponse(answer="I didn't catch that – which option would you like? You can choose from the options below.", used_context=docs or [])
1094
1095     # If we reach here, no explicit followup action matched – fallthrough to normal RAG flow
1096     # (This ensures user can ask a specific question after a followup prompt.)
1097     # Do not return here; allow normal RAG flow below.
1098
1099     # helper: detect if the user is asking for warranty/gst style queries and provide relevant context
1100     warranty_keywords = ["warranty", "warranties", "warranty policy", "warranty terms", "warranty claim"]
1101     gst_keywords = ["gst", "tax", "vat", "gst percentage", "gst percent", "gst %", "gst rate"]
1102
1103     # If the user asks about warranty or GST, try to extract from context first; if not found, use fallback
1104     if any(k in low_both for k in warranty_keywords + gst_keywords):
1105         # embed the user message and query for context
1106         try:
1107             query_vec = embed_query(normalized_user_msg)
1108         except Exception as e:
1109             print(f"[ERROR] Failed to embed user query for warranty/gst: {e}")
1110             return ChatResponse(answer=fallback(), used_context=[], from_fallback=True)
1111
1112         docs: List[str] = []
1113         docs.extend(_query_collection(end_customer_collection, "END_CUSTOMER", query_vec))
1114         docs.extend(_query_collection(spare_parts_collection, "SPARE_PARTS", query_vec))
1115         docs.extend(_query_collection(dealer_collection, "DEALERS", query_vec))
1116
1117         # Search docs for explicit warranty or gst info
1118         doc_combined = "\n\n".join(docs).lower()

```

```

1119         found_warranty = "warranty" in doc_combined or "warranty" in doc_combined
1120         # Defensive check for GST: treat presence of 'gst' or a percent sign as an in
1121         if isinstance(doc_combined, str):
1122             found_gst = ("gst" in doc_combined) or ("% " in doc_combined)
1123         else:
1124             found_gst = "gst" in doc_combined
1125
1126         answer_lines = []
1127         if any(k in low_both for k in warranty_keywords):
1128             if found_warranty:
1129                 # call the main model to produce a precise answer from context (honor
1130                 raw = generate_with_groq(context="\n\n---\n\n".join(docs), user_quest
1131                 if raw is None or "UNSURE_FROM_DATA" in raw:
1132                     answer_lines.append(DEFAULT_WARRANTY_TEXT)
1133                 else:
1134                     answer_lines.append(raw.strip())
1135             else:
1136                 answer_lines.append(DEFAULT_WARRANTY_TEXT)
1137         if any(k in low_both for k in gst_keywords):
1138             if found_gst:
1139                 # again prefer dataset context, but if not explicit use default
1140                 # quick heuristic: extract line(s) with "gst" or "%"
1141                 gst_matches = re.findall(r"([A-Za-z0-9 _/()%]+):\s*([^\n]*?(gst|%)["
1142                 if gst_matches:
1143                     gst_vals = [m[1].strip() for m in gst_matches]
1144                     answer_lines.append("GST information found: " + "; ".join(sorted(
1145                 else:
1146                     answer_lines.append(DEFAULT_GST_TEXT)
1147             else:
1148                 answer_lines.append(DEFAULT_GST_TEXT)
1149
1150         return ChatResponse(answer="\n\n".join(answer_lines), used_context=docs, from
1151
1152     # If not matched earlier, proceed to normal embedding and RAG
1153     try:
1154         query_vec = embed_query(normalized_user_msg)
1155     except Exception as e:
1156         print(f"[ERROR] Failed to embed user query with OpenAI: {e}")
1157         return ChatResponse(
1158             answer=fallback(),
1159             used_context=[],
1160             from_fallback=True,
1161         )
1162
1163     docs: List[str] = []
1164
1165     docs.extend(_query_collection(end_customer_collection, "END_CUSTOMER", query_vec)
1166     docs.extend(_query_collection(spare_parts_collection, "SPARE_PARTS", query_vec))
1167     docs.extend(_query_collection(dealer_collection, "DEALERS", query_vec))
1168
1169     if not docs:
1170         print("[CHAT] No relevant documents found in any index.")
1171         return ChatResponse(
1172             answer=fallback(),
1173             used_context=[],
1174             from_fallback=True,
1175         )
1176
1177     unique_docs = list(dict.fromkeys(docs))
1178     context = "\n\n---\n\n".join(unique_docs)
1179
1180     # brand-only helper (hardened to avoid greetings/stopwords and generic terms like
1181     def is_brand_only(text: str) -> bool:

```

```

1182     text = (text or "").strip()
1183     if not text:
1184         return False
1185     # Avoid treating simple stopwords or short replies as brand queries
1186     stopwords = {"hi", "hello", "hey", "yes", "no", "ok", "okay", "thanks", "than
1187     lowt = text.lower()
1188     if lowt in stopwords:
1189         return False
1190     # Generic nouns/phrases that should NOT be treated as brand names
1191     generic_blocklist = {
1192         "transport", "transportation", "transport details", "transportation detai
1193         "transport how", "transportation how", "shipping", "shipping details", "d
1194         "transportation?", "transport?", "how to transport", "how transport"
1195     }
1196     # If text contains any blocklisted generic term, it's not a brand-only query
1197     for g in generic_blocklist:
1198         if g in lowt:
1199             return False
1200     if re.search(r"\d", text):
1201         return False
1202     if len(text.split()) > 3:
1203         return False
1204     # If text is extremely short (1-2 chars) avoid brand-only
1205     if len(text) <= 2:
1206         return False
1207     # Also ensure the text looks like a brand token (contains letters)
1208     if not re.search(r"[A-Za-z]", text):
1209         return False
1210     return True
1211
1212     # If brand-only detection should consider translated English (so local-language b
1213     # we will test both original and translated.
1214     if is_brand_only(normalized_user_msg) or (low_en and is_brand_only(low_en)):
1215         brand = normalized_user_msg.strip()
1216         candidate_models: List[str] = []
1217         for d in unique_docs:
1218             pairs = re.findall(r"([A-Za-z0-9 _/()%~]+\s*([^\n]+)", d)
1219             for (k, v) in pairs:
1220                 k_clean = k.strip().lower()
1221                 val = v.strip()
1222                 if "model" in k_clean or "machine" in k_clean or "machine model" in k
1223                     if re.search(r"[A-Za-z]+\d", val) or re.search(r"\d+[A-Za-z]+", v
1224                         candidate_models.append(val)
1225         if not candidate_models:
1226             for d in unique_docs:
1227                 parts = re.split(r"[|,/\\n]+", d)
1228                 for p in parts:
1229                     token = p.strip()
1230                     if not token:
1231                         continue
1232                     if re.search(r"[A-Za-z]+\d", token) or re.search(r"\d+[A-Za-z]+",
1233                         candidate_models.append(token)
1234         seen = set()
1235         models = []
1236         for m in candidate_models:
1237             m_norm = m.strip()
1238             m_norm = re.sub(r"^[\\W_]+|[\\W_]+$", "", m_norm)
1239             if m_norm and m_norm not in seen:
1240                 if re.search(r"\bton\b", m_norm.lower()):
1241                     continue
1242                 if re.search(r"^(vj|sb)\b", m_norm, re.IGNORECASE):
1243                     continue
1244                 if m_norm.lower() == brand.lower():

```

```

1245         continue
1246         seen.add(m_norm)
1247         models.append(m_norm)
1248     if models:
1249         md_lines = [f"**Models for {brand}:**"]
1250         for m in models:
1251             md_lines.append(f"- {m}")
1252         md_answer = "\n".join(md_lines)
1253         return ChatResponse(
1254             answer=md_answer,
1255             used_context=unique_docs,
1256             from_fallback=False,
1257         )
1258
1259 # model (VJ) specific handling: return full rows for the model followed by a bull
1260 vj_match_user = re.search(r"\b(vj)[\s-]*?(\\d{1,3})(?:\\s*(hd))?\b", normalized_us
1261 if vj_match_user:
1262     # build normalized base token (e.g., 'vj20')
1263     parts = [vj_match_user.group(1) or "", vj_match_user.group(2) or ""]
1264     # user may have typed 'VJ20 HD' – capture that but we'll try matching logic b
1265     if vj_match_user.group(3):
1266         parts.append(vj_match_user.group(3))
1267     key_raw = "".join(parts)
1268     key_norm = re.sub(r"^[a-z0-9]", "", key_raw.lower())
1269
1270     # Candidate search order:
1271     # 1) exact token user asked (key_norm)
1272     # 2) if no exact match and user didn't ask HD, try key_norm + 'hd'
1273     # 3) if still none, try rows that contain key_norm anywhere (broader)
1274     # 4) fallback to collecting rows via the compatibility collector
1275     candidate_keys = [key_norm]
1276     if not key_norm.endswith("hd"):
1277         candidate_keys.append(key_norm + "hd")
1278     else:
1279         # if user asked VJXXHD explicitly, try without hd too
1280         candidate_keys.append(re.sub(r"hd$", "", key_norm))
1281
1282     src_rows: List[str] = []
1283     matched_key: Optional[str] = None
1284
1285     for cand in candidate_keys:
1286         cand_low = cand.lower()
1287         found = []
1288         for d in unique_docs:
1289             if cand_low in d.lower():
1290                 found.append(d)
1291         if found:
1292             src_rows = found
1293             matched_key = cand_low
1294             break
1295
1296     # If still no rows found, do a broader scan: any doc containing the base nume
1297     if not src_rows:
1298         broader = []
1299         for d in unique_docs:
1300             if key_norm in d.lower():
1301                 broader.append(d)
1302         if broader:
1303             src_rows = broader
1304             matched_key = key_norm
1305
1306     # If still empty, use the compatibility collector as last resort
1307     if not src_rows:

```

```

1308         compat_set, src_rows2 = _collect_all_compatible_machines_or_breakers(key_
1309         if src_rows2:
1310             src_rows = src_rows2
1311             # if compat_set contains a vj variant, pick the first vj variant as m
1312             chosen = None
1313             vj_variants = sorted([v for v in compat_set if re.fullmatch(r"vj\d+hd
1314             if vj_variants:
1315                 # prefer HD variant if exists
1316                 for vv in vj_variants:
1317                     if vv.endswith("hd"):
1318                         chosen = vv
1319                         break
1320                 if not chosen:
1321                     chosen = vj_variants[0]
1322             matched_key = chosen or key_norm
1323
1324         if not src_rows:
1325             # nothing found at all
1326             return ChatResponse(answer=fallback(), used_context=unique_docs, from_fal
1327
1328         # If matched_key is None, set default
1329         if not matched_key:
1330             matched_key = key_norm
1331
1332         # Determine display name (e.g., "VJ20 HD" or "VJ20")
1333         m_m = re.match(r"vj(\d{1,3})(hd)?", matched_key, flags=re.IGNORECASE)
1334         display_model = key_raw.upper()
1335         if m_m:
1336             num = m_m.group(1)
1337             hd_flag = m_m.group(2)
1338             display_model = f"VJ{num}" + (f" HD" if hd_flag else "")
1339
1340         # Merge all fields across rows to produce ONE consolidated set of key: value
1341         merged_fields = _merge_rows_to_field_map(src_rows)
1342
1343         # -----
1344         # FILTER: fields to hide for model-only responses
1345         hide_fields_when_model_only = {
1346             "sku", "stock available", "stock on hand", "warehouse name", "warehouse",
1347             "selling price", "selling price end customer", "selling price - end custo
1348             "spare parts", "spare part", "spare parts sku", "sparepart", "part sku",
1349             # synonyms-related keys to hide
1350             "synonyms of item name", "synonym", "synonyms", "synonyms of item name",
1351         }
1352         # -----
1353
1354         # Detect if the user requested a specific detail (priority)
1355         detail_map = {
1356             "price": ["price", "mrp", "price incl", "price incl gst", "price includin
1357             "sku": ["sku"],
1358             "stock": ["stock on hand", "stock", "stock available"],
1359             "warranty": ["warranty", "warranty period", "warranty terms"],
1360             "gst": ["gst", ""],
1361             "chisel": ["chisel", "chisel dia", "chisel dia ( mm )", "chisel dia ( mm
1362             "weight": ["weight", "breaker weight", "breaker weight ( kg )", "breaker
1363             "impact": ["impact", "impact in joules", "impact rate ( bpm )", "impact r
1364             "pin": ["pin size", "pin size ( mm )"],
1365             "primary application": ["primary application", "primary application:"],
1366             "machine": ["machine model", "machine", "machine brand"],
1367         }
1368         # find which detail user explicitly asked for by scanning the raw (lowercased
1369         requested_detail_key = None
1370         for dk in detail_map.keys():

```

```

1371         if dk in low_both:
1372             requested_detail_key = dk
1373             break
1374
1375 # helper to find a value in merged_fields by candidate fragments
1376 ✓ def _find_field_value_by_candidates(cands: List[str], field_map: Dict[str, str]):
1377     cand_lower = [c.lower() for c in cands]
1378     # prefer exact substring matches in key names
1379     for k in field_map:
1380         lk = k.lower()
1381         for cand in cand_lower:
1382             if cand in lk:
1383                 val = field_map.get(k)
1384                 if val:
1385                     return val
1386     # fallback: try to find any key where value contains typical signal for c
1387     return None
1388
1389 # If user requested a specific detail, try to obtain its value (prefer merged
1390 requested_detail_value = None
1391 requested_label = None
1392 if requested_detail_key:
1393     candidates = detail_map.get(requested_detail_key, [])
1394     # search merged fields
1395     v = _find_field_value_by_candidates(candidates, merged_fields)
1396     if v:
1397         requested_detail_value = v
1398         requested_label = candidates[0].title()
1399     else:
1400         # if 'price' requested, try _find_price_for_model across docs
1401         if requested_detail_key == "price":
1402             # try to query docs for matched_key model
1403             try:
1404                 query_vec = embed_query(matched_key)
1405                 docs_for_price: List[str] = []
1406                 docs_for_price.extend(_query_collection(end_customer_collection, query_vec))
1407                 docs_for_price.extend(_query_collection(spare_parts_collection, query_vec))
1408                 docs_for_price.extend(_query_collection(dealer_collection, query_vec))
1409                 price_found = _find_price_for_model(matched_key, docs_for_price)
1410                 if price_found:
1411                     requested_detail_value = price_found
1412                     requested_label = "Price"
1413                 else:
1414                     requested_detail_value = None
1415                     requested_label = "Price"
1416             except Exception as e:
1417                 print(f"[ERROR] Failed to embed/query for price priority: {e}")
1418                 requested_detail_value = None
1419                 requested_label = "Price"
1420         else:
1421             # not price and not in merged_fields
1422             requested_detail_value = None
1423             requested_label = requested_detail_key.title()
1424
1425 # Format details as bullet points (one block only), but put requested detail
1426 details_lines = [f"**Details for {display_model}**"]
1427
1428 # If requested detail exists, put it first
1429 if requested_detail_key:
1430     # If we have a value, show it; otherwise say not found in dataset for exp
1431     if requested_detail_value:
1432         details_lines.append(f"- {requested_label}: {requested_detail_value}")
1433     else:

```

```

1434         details_lines.append(f"- {requested_label}: Not found in dataset.")
1435
1436     # Build the rest of the details (excluding hidden fields and excluding field
1437     preferred_order = ["model", "breaker model", "sku", "brand", "machine brand",
1438     used_keys = set()
1439     # If requested detail came from a merged_fields key, exclude that key when li
1440     if requested_detail_key and requested_detail_value:
1441         for k, v in merged_fields.items():
1442             if v == requested_detail_value:
1443                 used_keys.add(k)
1444
1445     for k in preferred_order:
1446         for key in list(merged_fields.keys()):
1447             if k == key:
1448                 val = merged_fields.get(key)
1449                 if val:
1450                     # skip hidden keys
1451                     if key.lower() in hide_fields_when_model_only:
1452                         continue
1453                     # skip if already used
1454                     if key in used_keys:
1455                         continue
1456                     details_lines.append(f"- {key.title()}: {val}")
1457                     used_keys.add(key)
1458     # remaining keys (exclude hidden ones)
1459     for key in sorted(merged_fields.keys()):
1460         if key in used_keys:
1461             continue
1462         if key.lower() in hide_fields_when_model_only:
1463             continue
1464         val = merged_fields.get(key)
1465         if val:
1466             details_lines.append(f"- {key.title()}: {val}")
1467
1468     # Extract compatible machines (Brand - Model). Use dedicated parser to avoid
1469     model_keys = {matched_key, key_norm}
1470     compatible_machines = _parse_compatible_machine_tokens_from_rows(src_rows, al
1471
1472     # If parser gives nothing, also try scanning all unique_docs for any 'machine
1473     if not compatible_machines:
1474         fallback_rows = []
1475         for d in unique_docs:
1476             if matched_key in d.lower() or key_norm in d.lower():
1477                 fallback_rows.append(d)
1478         compatible_machines = _parse_compatible_machine_tokens_from_rows(fallback
1479
1480     compat_lines = ["\n**Compatible machines:**"]
1481     if compatible_machines:
1482         for cm in compatible_machines:
1483             compat_lines.append(f"- {cm}")
1484     else:
1485         compat_lines.append("No compatible machines listed in dataset.")
1486
1487     # Only add followup if below limit. Also add hidden machine-readable comment
1488     followup_text = ""
1489     if prior_followups < SALES_NUDGE_LIMIT:
1490         followup_text = (
1491             "\n\n**Would you like any of the following for this model?**\n"
1492             "- Brochure (reply 'brochure')\n"
1493             "- Connect me with a dealer/contact for a quote (reply 'connect')\n"
1494             "- Get a price / proforma (reply 'price')\n"
1495             "- More specs or warranty details (reply 'specs' or 'warranty')\n"
1496             "Reply with which option you'd like, or 'no' to skip."

```



```

1497         )
1498
1499     # Attach a hidden HTML comment that the front-end ignores but we can parse on
1500     hidden_comment = f"<!--MODEL_CTX:{matched_key}|OPTIONS:brochure,connect,price
1501
1502     reply_text = "\n\n".join(["\n".join(details_lines), "\n".join(compat_lines)])
1503
1504     # find brochure for matched_key (try matched_key then alt without/with hd)
1505     fname = BROCHURE_MAP.get(matched_key)
1506     if not fname:
1507         alt = re.sub(r"hd$", "", matched_key)
1508         fname = BROCHURE_MAP.get(alt)
1509         if not fname and not matched_key.endswith("hd"):
1510             fname = BROCHURE_MAP.get(matched_key + "hd")
1511
1512     brochure_url = None
1513     if fname:
1514         base = str(request.base_url).rstrip("/")
1515         brochure_url = f"{base}/brochures/view/{fname}"
1516
1517     return ChatResponse(answer=reply_text, used_context=src_rows or unique_docs,
1518
1519 # tonnage handling
1520 user_ton = _extract_ton_from_text(normalized_user_msg)
1521 if user_ton is not None:
1522     matched_breakers: Set[str] = set()
1523     matched_rows: List[str] = []
1524     for d in unique_docs:
1525         pairs = re.findall(r"([A-Za-z0-9 _/()%~+):\s*([^\n]+)", d)
1526         capacity_fields = []
1527         for (k, v) in pairs:
1528             if re.search(r"capacit|tonnage|ton\b|tons\b|class", k, re.IGNORECASE):
1529                 capacity_fields.append((k.strip(), v.strip()))
1530         if capacity_fields:
1531             for (k, v) in capacity_fields:
1532                 v_clean = v.replace("\u2013", "-").replace("\u2014", "-") # norm
1533                 # normalize "to" forms into hyphen for consistent parsing
1534                 v_clean = re.sub(r"\bto\b", "-", v_clean, flags=re.IGNORECASE)
1535                 # find ranges like 7-10 or 7 - 10, optionally with T/Ton
1536                 rmatches = re.findall(r"(\d{1,3})\s*(?:-|-|\s*)(\d{1,3})", v_clean)
1537                 matched = False
1538                 for rmatch in rmatches:
1539                     a = int(rmatch[0]); b = int(rmatch[1])
1540                     if a <= user_ton <= b:
1541                         matched_rows.append(d)
1542                         matched_breakers.update(_find_vj_tokens_in_text(d))
1543                         matched = True
1544                         break
1545                 if matched:
1546                     continue
1547                 # single number like '8 Ton' or '8T' inside capacity field
1548                 sm = re.search(r"\b(\d{1,3})\b", v)
1549                 if sm and int(sm.group(1)) == user_ton:
1550                     matched_rows.append(d)
1551                     matched_breakers.update(_find_vj_tokens_in_text(d))
1552             else:
1553                 # fallback: examine raw row text for range patterns
1554                 text_norm = d.replace("\u2013", "-").replace("\u2014", "-")
1555                 text_norm = re.sub(r"\bto\b", "-", text_norm, flags=re.IGNORECASE)
1556                 rmatches = re.findall(r"(\d{1,3})\s*(?:-|-|\s*)(\d{1,3})", text_norm)
1557                 for rmatch in rmatches:
1558                     a = int(rmatch[0]); b = int(rmatch[1])
1559                     if a <= user_ton <= b:

```

```

1560         matched_rows.append(d)
1561         matched_breakers.update(_find_vj_tokens_in_text(d))
1562         break
1563     if matched_breakers:
1564         lines = [f"**Breakers compatible with {user_ton} Ton machines:**"]
1565         for b in sorted(matched_breakers):
1566             lines.append(f"- {b.upper()}")
1567         followup_text = ""
1568         if prior_followups < SALES_NUDGE_LIMIT:
1569             followup_text = (
1570                 "\n\n**Would you like any of the following?**\n"
1571                 "- Brochure (reply 'brochure')\n"
1572                 "- Connect me with a dealer/contact for a quote (reply 'connect')\n"
1573                 "- Get a price / proforma (reply 'price')\n"
1574                 "- More specs or warranty details (reply 'specs' or 'warranty')\n"
1575                 "Reply with which option you'd like, or 'no' to skip."
1576             )
1577         # Hidden comment does not have a model key in this flow (generic), but in
1578         hidden_comment = "<!--MODEL_CTX:|OPTIONS:brochure,connect,price,specs-->"
1579         return ChatResponse(answer="\n".join(lines) + followup_text + "\n\n" + hi
1580
1581     # Fallback to LLM with context (RAG)
1582     raw = generate_with_groq(context=context, user_question=normalized_user_msg, user
1583     if raw is None:
1584         return ChatResponse(answer=fallback(), used_context=unique_docs, from_fallbac
1585
1586     raw = raw.strip()
1587
1588     if "UNSURE_FROM_DATA" in raw:
1589         return ChatResponse(answer=fallback(), used_context=unique_docs, from_fallbac
1590
1591     # try to detect if LLM output contains a VJ model; if so find brochure and add fo
1592     brochure_url = None
1593     m = re.search(r"\b(vj)[\s-]*?(\\d{1,3})(?:\\s*(hd|hd$))?\\b", normalized_user_msg,
1594     model_key_norm = None
1595     if m:
1596         parts = [m.group(1) or "", m.group(2) or ""]
1597         if m.group(3):
1598             parts.append(m.group(3))
1599         key_raw = "".join(parts)
1600         key_norm = re.sub(r"^[a-z0-9]", "", key_raw.lower())
1601         model_key_norm = key_norm
1602         fname = BROCHURE_MAP.get(key_norm)
1603         if not fname:
1604             alt_key = re.sub(r"hd$", "", key_norm)
1605             fname = BROCHURE_MAP.get(alt_key)
1606         if fname:
1607             base = str(request.base_url).rstrip("/")
1608             brochure_url = f"{base}/brochures/view/{fname}"
1609
1610     # Also check if the model is referenced in the raw LLM output (if no explicit men
1611     if not brochure_url:
1612         m2 = re.search(r"\b(vj)[\s-]*?(\\d{1,3})(?:\\s*(hd))?\b", raw, re.IGNORECASE)
1613         if m2:
1614             key_raw = "".join([m2.group(1) or "", m2.group(2) or ""] + ([m2.group(3)]
1615             key_norm = re.sub(r"^[a-z0-9]", "", key_raw.lower())
1616             fname = BROCHURE_MAP.get(key_norm)
1617             if fname:
1618                 base = str(request.base_url).rstrip("/")
1619                 brochure_url = f"{base}/brochures/view/{fname}"
1620                 model_key_norm = key_norm
1621
1622     final_answer = raw

```

```

1623
1624 # If we plan followups, add hidden comment to keep model context for the client's
1625 followups_text = ""
1626 hidden_comment = ""
1627 # Only add followups when the raw (LLM) response seems to mention a specific model
1628 if prior_followups < SALES_NUDGE_LIMIT and (model_key_norm or re.search(r"\b(vj)\b", raw_response)):
1629     followups_text = (
1630         "\n\n**Would you like any of the following for this model?**\n"
1631         "- Brochure (reply 'brochure')\n"
1632         "- Connect me with a dealer/contact for a quote (reply 'connect')\n"
1633         "- Get a price / proforma (reply 'price')\n"
1634         "- More specs or warranty details (reply 'specs' or 'warranty')\n\n"
1635         "Reply with which option you'd like, or 'no' to skip."
1636     )
1637     if model_key_norm:
1638         hidden_comment = f"<!--MODEL_CTX:{model_key_norm}|OPTIONS:brochure,connect,price,specs-->"
1639     else:
1640         # no explicit model key, keep generic hidden comment (client will handle)
1641         hidden_comment = "<!--MODEL_CTX:|OPTIONS:brochure,connect,price,specs-->"
1642
1643     if followups_text:
1644         final_answer = f"{final_answer}{followups_text}"
1645     if hidden_comment:
1646         final_answer = f"{final_answer}\n\n{hidden_comment}"
1647
1648 # If brochure_url found, also return it as a separate field (frontend will avoid
1649 return ChatResponse(answer=final_answer, used_context=unique_docs, from_fallback=

```